Lesson 1: Data Management and Analysis in Excel

Operations engineers spend a surprising amount of their working week in spreadsheets. The ticketing system spits out a Comma-Separated Values (CSV) file. The monitoring tool exports an uptime log. The cloud provider hands back a billing breakdown. A senior engineer asks, "What was our Mean Time To Resolve (MTTR) last month, and where did we breach Service Level Agreement (SLA)?" and the answer is sitting in three different files that nobody has joined together yet. The person who can open those files, clean them up, calculate the right numbers, and hand back a defensible answer is the person who becomes trusted with harder questions over time. This lesson is about being that person.

This course uses a running scenario to keep the work grounded. You are an operations analyst at **OrbitCo**, a mid-sized Software-as-a-Service (SaaS) company whose support team has been fielding a growing volume of incidents. Leadership has asked for a 30-day review: how many tickets came in, how quickly were they resolved, where are you losing time, and is uptime trending the way you promised in your SLA? You have two raw artifacts to work with: a **30-day ticket export** from the ticketing system (roughly 420 rows with columns for ticket ID, opened timestamp, closed timestamp, severity, team, and resolution code) and an **uptime log** recording every minute of downtime across three production services during the same period. These are messy, real-looking exports — duplicates, inconsistent date formats, a handful of missing severity values, and a stray merged-header row that someone pasted in by mistake. Every lesson in this course uses this OrbitCo dataset. By the end of this lesson, you will have a cleaned, analyzable workbook that becomes the data foundation for the dashboard work that follows.

Use of productivity tools (Excel, PowerPoint) for technical reporting is a core Registered Training Institute (RTI) topic for this course, and this lesson focuses on the Excel half. You will leave with five things: a mental model for how to organize a technical workbook, a reliable procedure for cleaning a raw operational export, a working vocabulary of the formulas that do 90% of day-to-day operations analysis, fluency with Excel Tables and structured references, and the conditional-formatting reflexes that turn a wall of numbers into something your eye can actually read.

Learning Objectives

By the end of this lesson, you will be able to:

1. Organize operational data in Excel using appropriate worksheet structure, naming, and data types
2. Apply data validation and cleanup techniques to prepare raw operational exports for analysis
3. Use Excel formulas to calculate common technical metrics (MTTR, uptime percentage, ticket counts by severity)
4. Use Excel Tables and structured references to build spreadsheets that stay correct as data grows
5. Apply sorting, filtering, and conditional formatting to surface anomalies in operational data

---

Spreadsheet Design for Technical Data

A technical workbook is a small software project. If you build it the way most people build their first spreadsheet — everything piled onto one tab, totals embedded in the middle of the data, fonts and colors doing the work that formulas should be doing — it will survive exactly one edit before it starts producing wrong answers. The conventions below are not stylistic preferences. They are the difference between a workbook that can be trusted next month and one that has to be thrown away and rebuilt.

Worksheet Organization: Raw, Processed, and Outputs

The first discipline is separating **raw data**, **processed data**, and **outputs** onto different worksheets. Raw is whatever came out of the source system, untouched. Processed is where you clean, type-convert, and enrich it. Outputs are the pivots, Key Performance Indicator (KPI) summaries, and charts that readers consume.

For the OrbitCo workbook, that means a `tickets_raw` tab holding the 420 rows exactly as they came from the ticketing system, a `tickets_clean` tab where you remove duplicates and normalize dates, a `uptime_raw` tab with the downtime log, a `uptime_clean` tab with computed minutes-of-downtime, and an `outputs` tab where the KPI table for the review lives. This layered structure matters because raw data is evidence. If a reviewer questions a number, you can always walk it back to the original export and prove the lineage. If you edited the raw tab in place, that chain is broken.

Naming Conventions and Named Ranges

Use **named ranges** and **table names** instead of cell references wherever a value will be referred to more than once. A formula that reads `=SUMIFS(Tickets[ResolutionMinutes], Tickets[Severity], "High")` is self-documenting; a formula that reads `=SUMIFS(D2:D421, G2:G421, "High")` is a booby trap. Six months from now, when someone inserts a column, the second formula silently starts summing the wrong data. The first formula keeps working.

Name your tables with short, purposeful names: `Tickets`, `Uptime`, `Severities`. Avoid the auto-generated `Table1`, `Table2` — those tell the next reader nothing.

Column Conventions

One fact per column. One record per row. A header row in row 1 with plain-text names. No merged cells inside a data region, ever. No blank columns or rows separating sections of the same dataset — use Tables to bound a dataset instead.

Common Mistakes

Three patterns show up in nearly every hand-built operations workbook and each one breaks analysis:

- **Multi-header tables** — two rows of headers where the second row contains sub-categories. Pivots cannot read these. Replace with a single header row and a category column.
- **Embedded totals** — a "Total" row sitting in the middle of the data. It throws off every sort, filter, and pivot. Put totals on the outputs tab, not inside the data.
- **Color used as data** — rows highlighted red to mean "SLA breach" with no underlying column recording that fact. Color is a render; it must be backed by a real column or the information is invisible to formulas.

Data Entry, Validation, and Cleanup

Raw operational exports are almost never analysis-ready. The OrbitCo ticket export, for example, has three problems you will hit on every real job: the opened-timestamp column came in as text on some rows and as a proper datetime on others, there are seven duplicate ticket IDs caused by a reopen-bug in the ticketing system, and nineteen rows have a blank severity because the customer-service agent resolved the ticket before triaging it. Cleanup is a repeatable procedure, not an art.

Importing and Typing the Data

Use **Data → From Text/CSV** for CSV imports rather than copy-paste. The import dialog lets you explicitly set each column's data type, which prevents the classic failure where a ticket ID like `000432` is read as the number 432 and loses its leading zeros.

When a column arrives as text that should be a date, select the column and use **Data → Text to Columns → Delimited → Finish** with the target format set to Date — MDY (month-day-year) or DMY (day-month-year), depending on the source. For the OrbitCo export, the `OpenedAt` and `ClosedAt` columns both need this treatment.

Finding and Handling Duplicates

Data → Remove Duplicates deduplicates a range based on the columns you select. For tickets, dedupe on `TicketID` — not on the entire row, because the ticketing system's reopen-bug creates duplicates with slightly different timestamps, and a full-row dedupe would let them all through. Before you run it, copy the data to a scratch tab so you can compare row counts. You should lose exactly seven rows. If you lose more, the dedupe key is wrong.

Handling Missing Values

Missing values are a choice, not an accident. You can delete the row (acceptable if the record is meaningless without the field), impute a value (acceptable only if you document it), or flag the record and keep it. For operations analysis, **flagging** is usually safest. Add a helper column `SeverityIsMissing` that evaluates `=ISBLANK([@Severity])` and excludes those rows from severity-based metrics without hiding that they existed.

For the OrbitCo workbook, the nineteen missing-severity tickets should be flagged and the KPI table should report "Tickets with missing severity: 19" as its own line. That is honest reporting. Silently excluding them is not.

Data Validation Rules

Data Validation (Data → Data Validation) lets you restrict what can be entered into a cell. Use it on columns where the set of legal values is small and known: `Severity` (High/Medium/Low), `Team` (Platform/Customer/Infra), `ResolutionCode` (Fixed/Duplicate/WontFix/Other). A dropdown list prevents the typos and case inconsistencies — "high", "High ", "HIGH" — that break `COUNTIF` later.

Common Operational Data Sources

You will meet the same three export types over and over in operations work: **ticketing reports** (incidents, service requests, problems — usually joined with timestamps and severity), **monitoring CSVs** (metric samples with timestamps for CPU, latency, error rates), and **log exports** (event streams with timestamps, sources, and message text). Each has its own shape. A clean workflow looks the same regardless of source: import, type, dedupe, validate, flag, then analyze.

Activity: Clean the OrbitCo Incident Export

In this hands-on lab, you open a messy 420-row ticket export, walk through a defensible cleanup procedure (import, trim, case-normalize, validate, flag, dedupe), and produce a named `Tickets` Table that is ready for analysis. The goal is the repeatable discipline, not just a clean file — you should be able to explain every step when a reviewer asks.

Formulas for Technical Metrics

The formulas in this section calculate maybe 90% of the numbers an operations analyst produces in the course of a month. They are not impressive on paper and they are not clever. They are fluent. A good operations analyst writes them without hesitation and without bugs.

Arithmetic Basics

`SUM`, `AVERAGE`, `COUNT`, `MIN`, `MAX` — the five aggregators. `COUNT` counts numeric values; `COUNTA` counts any non-blank value (use `COUNTA` for columns of text like ticket IDs). On OrbitCo data, `=COUNTA(Tickets[TicketID])` gives you total ticket volume; `=AVERAGE(Tickets[ResolutionMinutes])` gives you raw MTTR in minutes.

Conditional Aggregation

`COUNTIF` counts cells matching a criterion. `SUMIF` and `AVERAGEIF` are the same idea for sum and average. For multi-criterion versions, use `COUNTIFS`, `SUMIFS`, and `AVERAGEIFS`.

For the OrbitCo KPI table, these two are your workhorses:

- Count of High-severity tickets: `=COUNTIF(Tickets[Severity], "High")`
- Average resolution time for High-severity tickets only: `=AVERAGEIF(Tickets[Severity], "High", Tickets[ResolutionMinutes])`

If you find yourself writing a formula with seven `IF`s nested together, stop — you want `COUNTIFS` or `SUMIFS`.

Decision Logic: IF, Nested IF, and IFS

`IF(condition, value_if_true, value_if_false)` is the basic branching formula. `IFS` is a newer, cleaner alternative for multi-branch logic: `=IFS(test1, val1, test2, val2, TRUE, fallback)`.

A common operational use is labeling tickets as SLA-breach or not:

```
=IF([@ResolutionMinutes] > [@SLAMinutes], "BREACH", "OK")
```

This adds a computed `SLAStatus` column to the Tickets table. From there, `=COUNTIF(Tickets[SLAStatus], "BREACH")` gives you the breach count. Note how the cleanup and the formula compound: a single new column feeds many downstream calculations.

Lookups: VLOOKUP and XLOOKUP

A **lookup** finds a value in a table based on a key. The classic function is `VLOOKUP(lookup_value, range, column_index, [FALSE])`. It searches left-to-right, requires the key to be in the first column, and returns by column position. Every one of those constraints has been a source of bugs for twenty years.

`XLOOKUP(lookup_value, lookup_array, return_array, [if_not_found])` replaces it. It defaults to exact match, it works left or right, and it returns by array, so inserting a column does not break it. On modern Excel, use `XLOOKUP`.

At OrbitCo, the SLA table lives on a separate `sla_lookup` tab with two columns: `Severity` and `SLAMinutes` (High = 60, Medium = 240, Low = 1440). To pull the right SLA target onto each ticket row:

```
=XLOOKUP([@Severity], SLALookup[Severity], SLALookup[SLAMinutes],
"Unknown")
```

Now the Tickets table has the per-ticket SLA, the `SLAStatus` formula works, and changing the SLA policy is a one-cell edit on the lookup tab.

MTTR, Uptime Percentage, and Ticket Volume by Severity

The three operational metrics this course keeps coming back to are all one-line formulas on the cleaned data:

- **Mean Time To Resolve (MTTR)**, in minutes: `=AVERAGE(Tickets[ResolutionMinutes])`. For severity-scoped MTTR, use `AVERAGEIF`.
- **Uptime percentage** over the 30-day window: `=1 - (SUM(Uptime[DowntimeMinutes]) / (30*24*60))`. Format the cell as a percentage with two decimal places. A month with 43 minutes of total downtime comes out at 99.90%.

- **Ticket volume by severity**: three `COUNTIF` calls on the Severity column, plus one `=COUNTBLANK(Tickets[Severity])` for the missing-severity count you flagged earlier.

Tables and Structured References

Everything above gets dramatically more reliable when you put the data inside an **Excel Table**. A Table is a bounded, named object that Excel treats as a first-class entity. Formulas that refer to it use structured references instead of cell coordinates, and those references update automatically as the Table grows or shrinks.

Creating a Table

Select any cell inside the data range and press **Ctrl+T** (or **Cmd+T** on Mac). Excel draws a box around the contiguous region, asks whether the first row is a header, and converts the range to a Table. Give it a meaningful name using the **Table Design** tab — for the OrbitCo workbook, rename `Table1` to `Tickets` immediately.

What Changes When You Convert to a Table

- Filter dropdowns appear automatically in the header row.
- Formulas entered in one cell of a Table column auto-fill down the entire column.
- New rows added at the bottom inherit formulas, formatting, and data validation.
- Named references like `Tickets[Severity]` become valid anywhere in the workbook.
- PivotTables and charts built from the Table automatically pick up new rows on refresh.

Structured References

Instead of `B2:B421`, you write `Tickets[Severity]`. Instead of `D2` from inside another formula, you write `[@ResolutionMinutes]` (the `@` means "this row").

A full example — an `SLAStatus` column calculated inside the Tickets Table:

```
=IF([@ResolutionMinutes] > [@SLAMinutes], "BREACH", "OK")
```

A full example of a structured reference used from a different sheet:

```
=COUNTIFS(Tickets[Severity], "High", Tickets[SLAStatus], "BREACH")
```

This counts the high-severity SLA breaches. When the Tickets Table grows from 420 rows to 580, the formula still works with no edits.

Why Tables Matter for Operations Reports

Operational data grows every day. A workbook built with hardcoded ranges breaks every time someone pastes in a new week of tickets. A workbook built with Tables and structured references survives that without any formula edits. This is the single most important reliability practice in this lesson, and it is the reason the On-the-Job Learning (OJL) 5d competency — developing and maintaining technical documentation, Standard Operating Procedures (SOPs), and runbooks — is tagged here. A Tables-based workbook can be used as living operations documentation, not a one-off snapshot.

Naming for Readability

In a workbook with three Tables — `Tickets`, `Uptime`, `SLALookup` — every formula reads like a sentence. In a workbook with `Table1`, `Table2`, `Table3`, every formula reads like a puzzle. Name your Tables the moment you create them. The five seconds of typing pays for itself the first time someone else opens the file.

Activity: Build the OrbitCo KPI Summary

In this hands-on lab, you extend the cleaned workbook with an `sla_lookup` tab and an `outputs` KPI summary driven entirely by structured-reference formulas. By the end you will be able to compute ticket volume, MTTR, SLA breach counts, and uptime percentage from the raw tables with no hardcoded ranges, so the workbook keeps working when new data lands next week.

Sorting, Filtering, and Conditional Formatting

The final piece of this lesson is turning a clean, structured dataset into something a human can read at a glance. Sorting, filtering, and conditional formatting are the three tools that surface anomalies without requiring the reader to compute anything in their head.

Sorting

Single-column sort is one click on the filter dropdown. Multi-column sort is **Data → Sort** with an Add Level button. For the Tickets Table, the most useful sort is "Severity ascending, then ResolutionMinutes descending" — it puts High severity at the top and, within that, the slowest-resolved tickets at the top. Those are the tickets worth looking at first.

Filtering

AutoFilter dropdowns appear on every Table by default. **Slicers** (Insert → Slicer) give you a button-style filter interface that is easier to use during a live walkthrough than dropdowns. For a workbook that a non-analyst will explore, slicers pay off immediately.

A common filter for incident review: filter to `Severity = High AND SLAStatus = BREACH`. That view is the agenda for the next team retrospective.

Conditional Formatting

Conditional formatting applies a visual rule based on a formula or value. The four most useful rule types for operations work:

- **Highlight Cells Rules** (e.g., cells greater than 60) — simple threshold checks.
- **Top/Bottom Rules** — highlight the top 10 values or bottom 10%.
- **Data Bars** — in-cell horizontal bars proportional to the value; good for comparing a long column of numbers.
- **Color Scales** — two- or three-color gradients; good for heat-mapping latency or error-rate grids.

For the OrbitCo workbook, one rule earns its keep immediately: a conditional format on the Tickets Table that highlights any row where `SLAStatus = BREACH` in red and `Severity = High` in bold. With this rule in place, a reader scrolling the sheet sees the problem rows without running any filters.

Rule Order and Precedence

Conditional formatting rules stack. The rule at the top of the Rules Manager wins ties. If you have a "Severity High → bold" rule and a "SLAStatus BREACH → red fill" rule, and a row matches both, both formats apply as long as they do not conflict. If they do conflict (two fill colors), order matters. Use **Home → Conditional Formatting → Manage Rules** to see and reorder them.

Wrap-Up

A maintainable workbook has layered worksheets, named Tables, structured references, validated inputs, and conditional formats that flag anomalies at a glance. The OrbitCo workbook you built in this lesson — `tickets_clean`, `uptime_clean`, and an `outputs` KPI summary — is the data foundation for the dashboard work that follows in the next lesson of this module.

Key Takeaways

- A technical workbook is a small software project: layer raw, processed, and outputs onto separate sheets so every number can be traced back to its source.
- Cleanup is a repeatable procedure — import with types, trim and normalize casing, validate legal values, flag missing data (do not silently drop it), then dedupe on the right key.
- The operational formula toolkit is small and fluent: `COUNTIF`/`SUMIF`/`AVERAGEIF`, `IF`/`IFS`, and `XLOOKUP` cover most of what an operations analyst writes in a given month.
- Convert every data range to an Excel Table and use structured references like `Tickets[Severity]`. The workbook will keep working when new rows arrive instead of silently going wrong.
- Sorting, filtering, and conditional formatting exist to let the tool do the scanning your eyes should not have to — a single "SLA breach → red" rule on a Table column surfaces the rows that need attention.

Reflection Questions

1. Think about the last spreadsheet you received from a colleague. Could you walk every KPI back to the raw data, or were some numbers floating free? What would you change about the layout?
2. When a reviewer challenges a number you reported — say, the MTTR on a monthly review — how would your current workbook hold up? Which of the disciplines in this lesson would strengthen your answer?
3. Missing-severity tickets: delete, impute, or flag? Pick one and defend the choice in one sentence for a specific operational reporting audience.